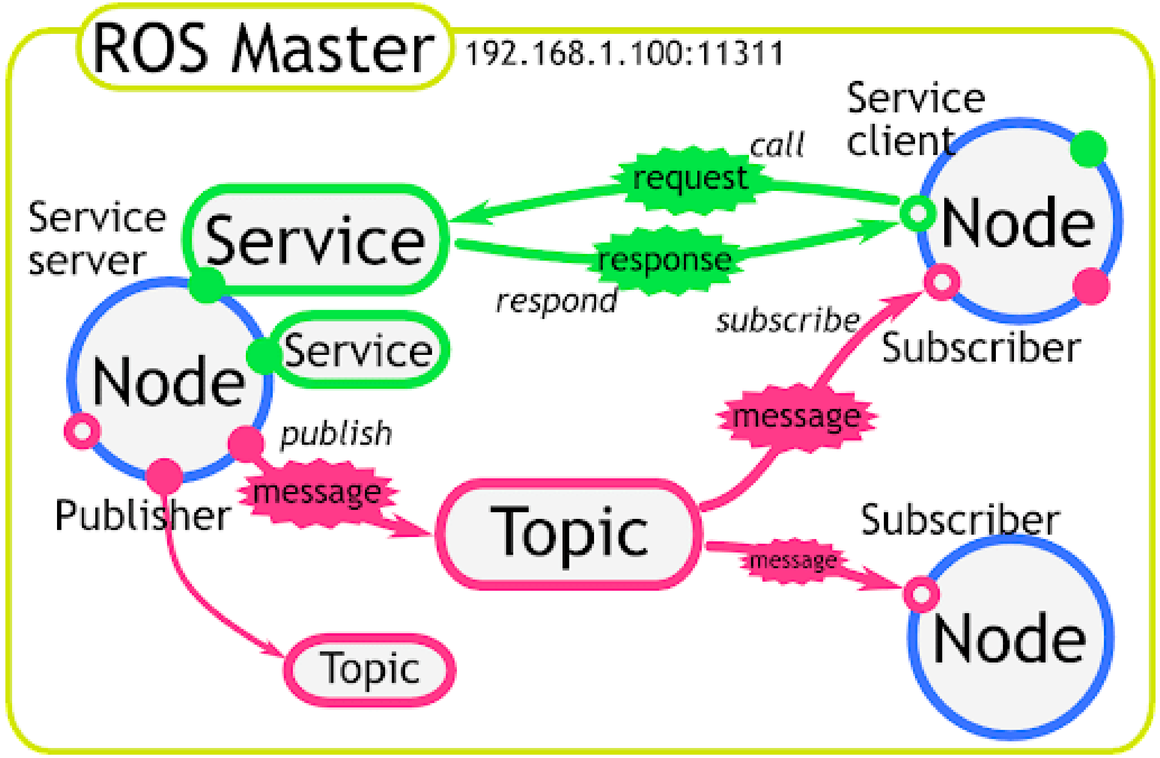
Robot Operating System(ROS)

Content

- Introduction to Linux and command line
- Introduction to ROS and installation
- ROS Core Concepts
  - Master
  - Nodes
  - Topics
  - Message
  - Services
- **ROS Lab 1**
- Create ROS WorkSpace and Packages.
- Write a ROS Publisher and Subscriber in python
- Write ROS Service and Client in Python
- Create Custom ROS Message
- ROS Launch file and RQT tool
- rosed edit files
- ROS bag file
- ROS projects

ROS Core Concepts

- ROS Master
- Nodes and Topics
- Messages
- Services
- Parameters
- Stacks and packages



More info
http://wiki.ros.org/ROS/Concepts

ROS Master

The ROS Master is a core component of the ROS system, responsible for maintaining a registry of all the nodes in the system and coordinating communication between them. It acts as a name server for nodes and allows nodes to find each other and communicate with each other using a publish-subscribe model. The ROS Master is typically run on the computer that is connected to the robot, and all other nodes in the system connect to it to register themselves and discover other nodes.

- The role of the Master is to enable individual ROS nodes to locate one another. Once these nodes have located each other they communicate with each other peer-to-peer.
- The Master also provides the Parameter Server.
- The Master is most commonly run using the roscore command, which loads the ROS Master along with other essential components.

Start a master with

```
> roscore
```

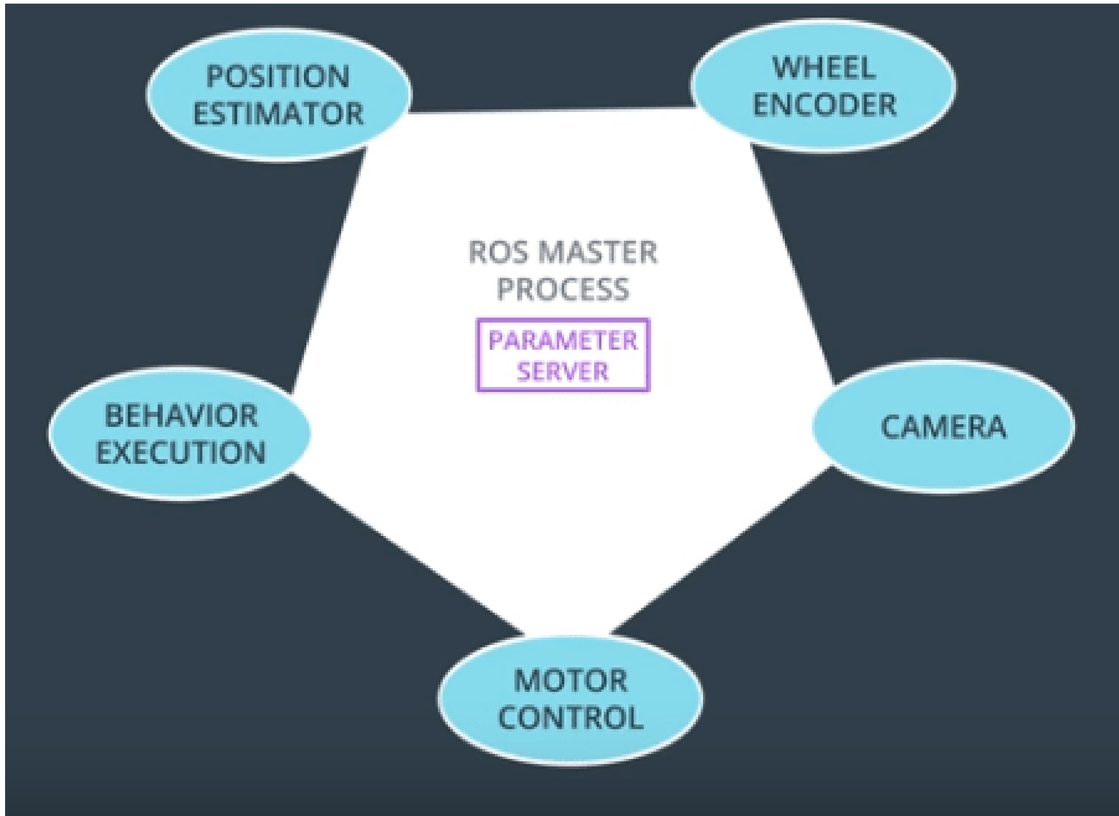
More info
http://wiki.ros.org/Master

ROS Paramter Service

The ROS parameter server is a component of the ROS system that allows nodes to store and retrieve parameters at runtime. These parameters can include things like configuration settings, calibration data, or tuning parameters for the robot's controllers.

The parameter server allows nodes to share data without having to explicitly pass it as input or output to a node. This can be useful for cases where a node may need to change its behavior based on the value of a parameter, or if a parameter needs to be changed during runtime to adjust the robot's behavior.

- ROS Master host Parameter server, it used to store parameter and configuration values and shared between running nodes.
- Example: mobile robots have a radius, nodes can be used to estimate position, and maybe different nodes are used to calculate speed.



ROS Node

In ROS, a node is a single process that performs a specific task. Nodes can be written in a variety of programming languages, such as C++ or Python, and communicate with other nodes through the ROS messaging system. Each node communicates with other nodes by publishing or subscribing to topics, or by providing or using services.

ROS nodes are the basic building blocks of a robot application, and can perform a wide variety of tasks, such as sensor processing, control, or visualization. For example, a robot might have a node that reads data from a sensor and publishes it on a topic, and another node that subscribes to that topic and uses the sensor data to control the robot's movement.

Each node in a ROS system must have a unique name, and nodes register themselves with the ROS Master when they start up. This allows other nodes to discover and communicate with them, even if they are running on different machines.

- It is an executable program running inside your application.
- You will write many nodes and put them into packages.

Run a node with

```
> rosrun package_name node_name
```

See active nodes with

```
> rosnode list
```

More info

http://wiki.ros.org/rosnode

ROS
Robot Operating System

Publish String Message

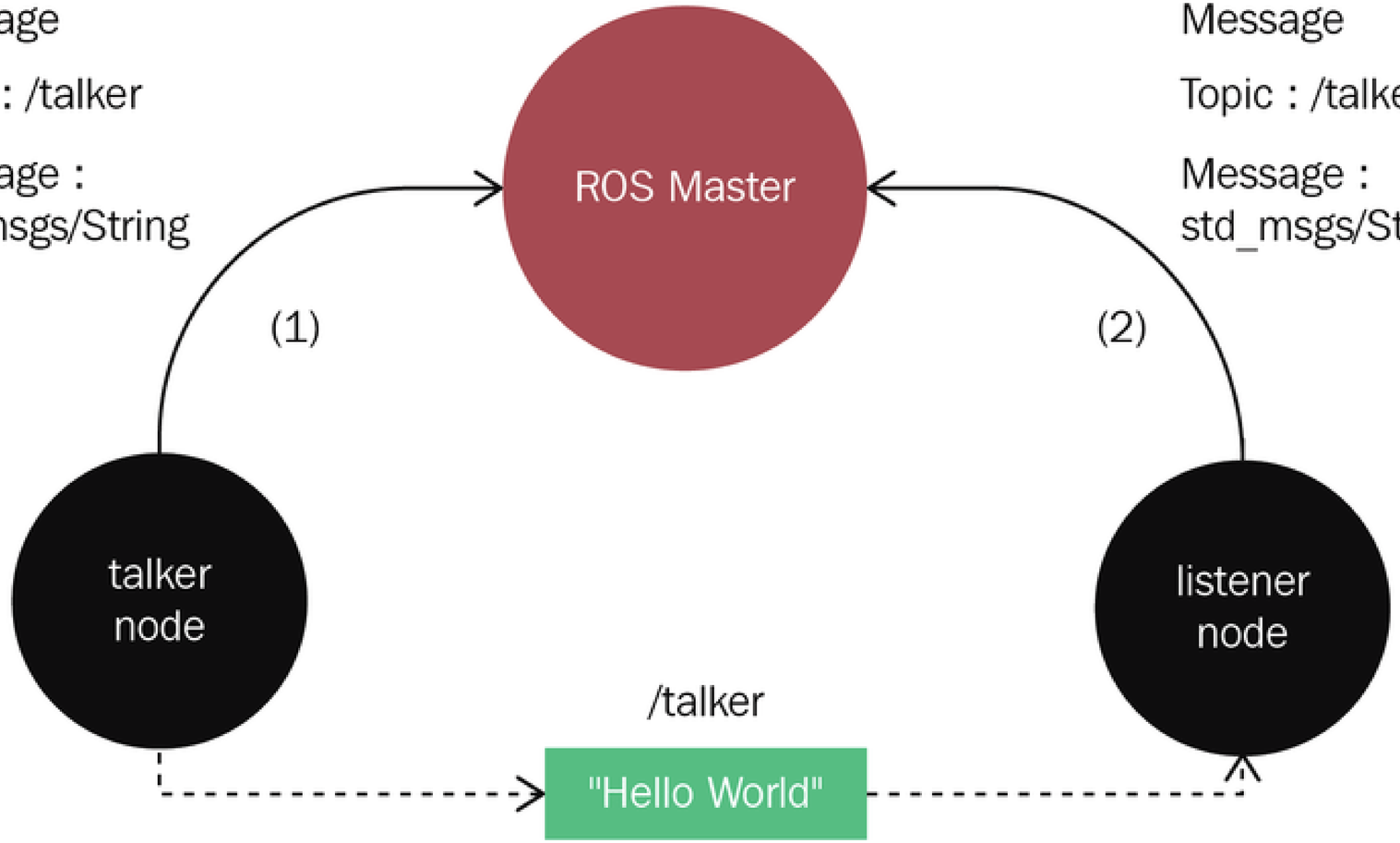
Topic : /talker

Message : std_msgs/String

Subscribe String Message

Topic : /talker

Message : std_msgs/String



ROS Topics

In ROS, a topic is a named channel over which nodes can send and receive messages. Nodes can publish messages to a topic, and other nodes can subscribe to that topic to receive the messages. This allows nodes to communicate with each other asynchronously, without the need for direct point-to-point communication.

Topics are used to transmit data that changes over time, such as sensor readings or control commands. For example, a robot might have a node that reads data from a camera and publishes it on a topic called "camera/image", and another node that subscribes to that topic and uses the image data for object recognition.

Each topic has a specific message type associated with it, which defines the structure of the messages that are exchanged on that topic. ROS provides a wide variety of built-in message types, such as std_msgs/String, std_msgs/Float32, and sensor_msgs/Image, as well as the ability to define custom message types.

- Ros topics transport information between nodes.
- This information is organized as a data structure and can have different data types.
- A topic is a name for a stream of messages with a defined type
- Nodes communicate with each other by publishing messages on topics
- Publish/Subscribe model: 1-to-N broadcasting

See active topics with

```
> rostopic list
```

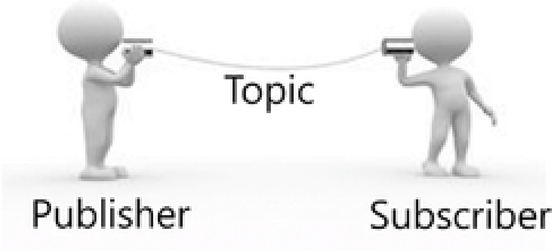
Subscribe and print the contents of a topic with

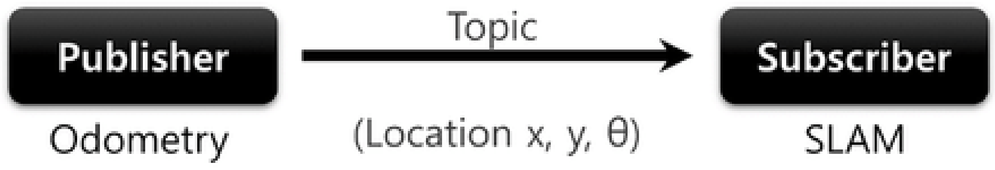
```
> rostopic echo /topic
```

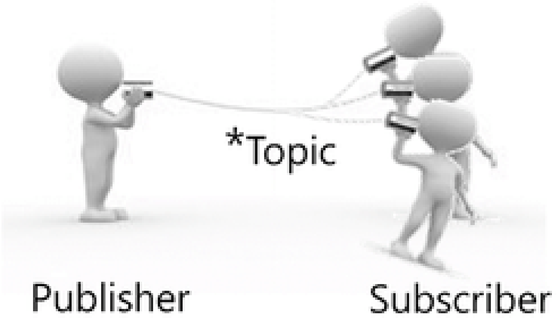
More info

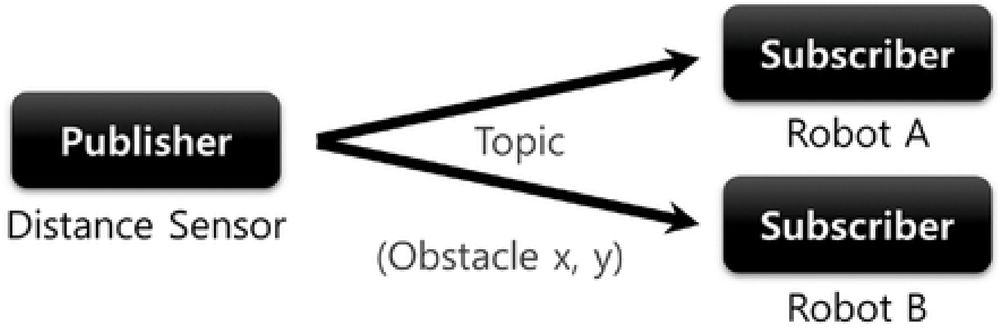
http://wiki.ros.org/rostopic

ROS
Robot Operating System









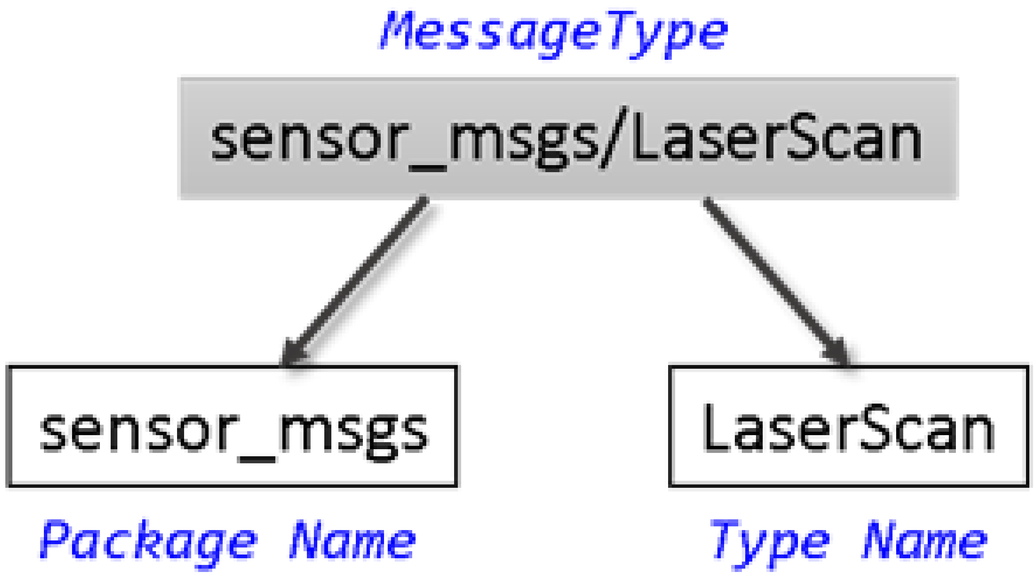
*Topic not only allows 1:1 Publisher and Subscriber communication, but also supports 1:N, N:1 and N:N depending on the purpose.

ROS Messages

- Nodes can process and share information through topics. The information they pass through these topics needs to be structured in some way, to make it actually usable. Such a structure is known as a data structure. In ROS, we can abstract these data structures as ROS message types. Common data structures in ROS are for example floats, integers, and strings.

- There are a lot of type messages in ROS, with over 200 different message types.

- Defined in *.msg files



Show the type of a topic

```
> rostopic type /topic
```

Publish a message to a topic

```
> rostopic pub /topic name
```

More info

http://wiki.ros.org/Messages

Building .msg Files

ROS
Robot Operating System

Package.xml

```
<build_depend>message_generation</build_depend>
<run_depend>message_runtime</run_depend>
```

At build time, we need "message_generation"

While at runtime, we only need "message_runtime"

CMakeLists.txt

```
find_package(catkin REQUIRED COMPONENTS roscpp rospy std_msgs message_generation)
```

```
catkin_package(
  CATKIN_DEPENDS
message_runtime ...
  )
```

```
add_message_files(
  FILES
  Num.msg
)
```

```
generate_messages(
  DEPENDENCIES
  std_msgs
)
```

More info

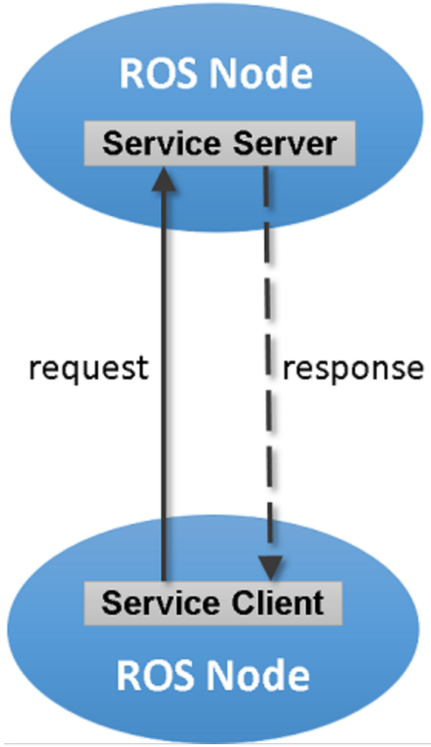
http://wiki.ros.org/Messages

ROS Services

A ROS service is a way for nodes in a ROS system to request and receive information from other nodes. Services are defined using the ROS service definition language and are similar to messages in that they are simple data structures, but they include a request and a response message.

A service request is sent by a client node to a server node, and the server node responds with a service response. The request and response messages have specific fields defined in the service definition file.

- Communication on service is bidirectional synchronous communication between the service client requesting a service and the service server responding to the request.
- Unlike topics, services have no publishers or subscribers.



List all the services that are currently available.

```
> rosservice list
```

Print information about specified services.

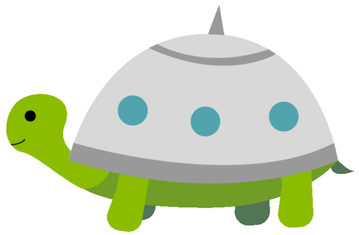
```
> rosservice info
```

DIRECTOR
STAGE MANAGER
Disney+

ROS Lab

Turtlesim Robot

Turtlesim is a simple ROS simulation that is included with the ROS installation. It is a 2D simulation of a turtle robot that can be controlled using ROS messages and services. The turtlesim node listens to ROS commands and updates the position and orientation of the turtle on the screen.

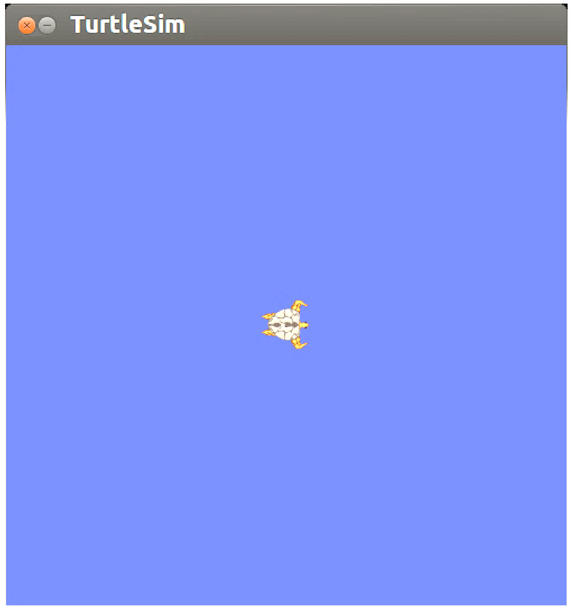


The turtlesim node can be launched using the following command:

```
$ roscore
```

```
$ rosrun turtlesim turtlesim_node
```

This will open a window showing the turtle in a 2D world. You can use the arrow keys to move the turtle around in the simulation.



ROS Nodes with Turtlesim

```
$ rosnode list
```

/rosout
/turtlesim

```
$ rosnode info /turtlesim
```

Node [/turtlesim]
Publications:
(This information is sent to nodes listening to /turtlesim)
* /turtle1/color_sensor [turtlesim/Color] (Color message in turtlesim package)
* /rosout [rosgraph_msgs/Log]
* /turtle1/pose [turtlesim/Pose]
(Pose message in turtlesim package for /turtle1)
Subscriptions:
* /turtle1/cmd_vel [unknown type] (This node will listen for command velocities)

ROS Service with Turtlesim

```
$ rosservice info /turtlesim
```

*** /turtle1/teleport_absolute**
* /turtlesim/get_loggers
* /turtlesim/set_logger_level
* /reset
* /spawn
* /clear
* /turtle1/set_pen
* /turtle
* /turtle1/set_pen
*** /turtle1/teleport_relative**
* /kill

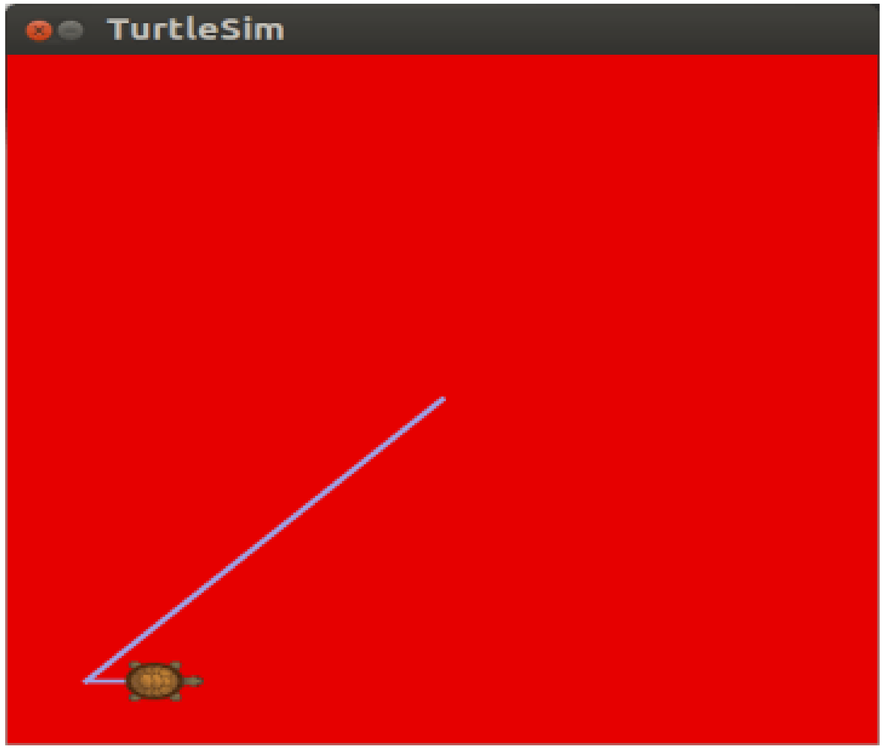
ROS Services to Move the Turtle

```
$ rosservice call /turtle1/teleport_absolute 1 1 0
```

```
$ rosservice call /turtle1/teleport_relative 1 0
```

ROS
Robot Operating System

Turtle After Absolute Move



Turtle After Relative Move

Turtlesim Node Topic Pose

```
$ rostopic info /turtle1/pose
```

This displays the message type.

```
Type: turtlesim/Pose
Publishers:
* /turtlesim (http://D104-45931:42032/)
Subscribers: None
```

```
$ rostopic type /turtle1/pose
```

Confirming the message type

```
turtlesim/Pose
```

```
$ rosmsg show turtlesim/Pose
```

Show/display the message fields.

```
float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
```

```
$ rostopic echo /turtle1/pose
```

```
x: 2.0
y: 1.0
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
---
x: 2.0
y: 1.0
theta: 0.0
linear_velocity: 0.0
angular_velocity: 0.0
```

Echo the message to the terminal, i.e. display the message values.

```
$ rosnode info /turtlesim
```

```
Node [/turtlesim]
Publications:
* /turtle1/color_sensor [turtlesim/Color]
* /rosout [rosgraph_msgs/Log]
* /turtle1/pose [turtlesim/Pose]
Subscriptions:
* /turtle1/cmd_vel [unknown type]
Services:
* /turtle1/teleport_absolute
* /turtlesim/get_loggers
* /turtlesim/set_logger_level
* /reset
* /spawn
* /clear
* /turtle1/set_pen
* /turtle1/teleport_relative
* /kill
```

```
$ rostopic type /turtle1/cmd_vel
```

geometry_msgs/Twist

```
$ rosmsg show geometry_msgs/Twist
```

geometry_msgs/Vector3 linear
float64 x
float64 y
float64 z
geometry_msgs/Vector3 angular
float64 x
float64 y
float64 z

Let us take a look at the geometry_msgs/Twist message type

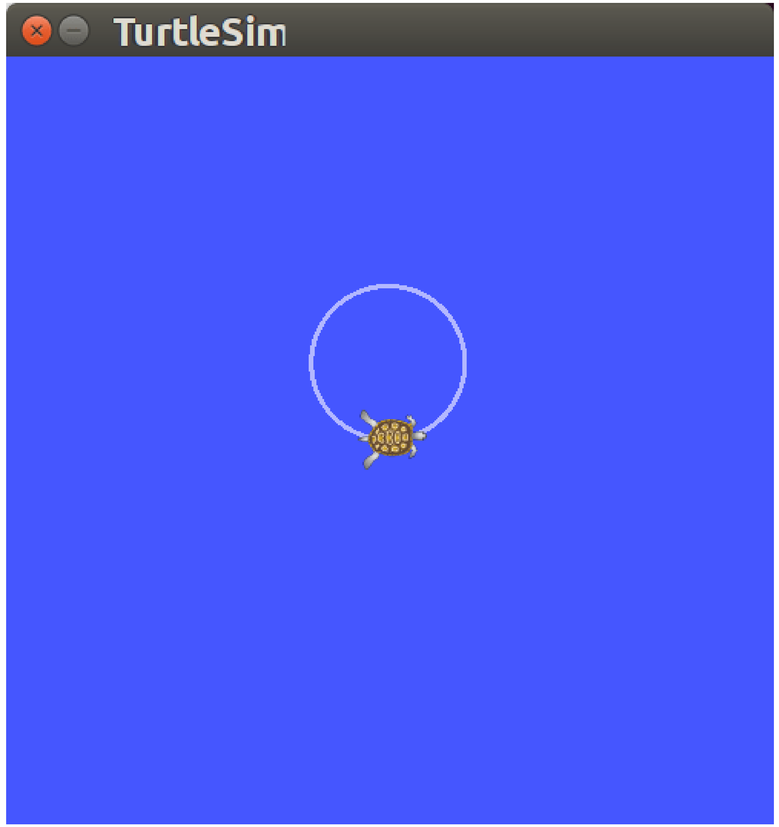
Moving the Turtle Once

```
$ rostopic pub -1 /turtle1/cmd_vel geometry_msgs/Twist -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 0.0]'
```

```
$ rostopic pub -1 /turtle1/cmd_vel geometry_msgs/Twist "linear:
    x: 0.0
    y: 0.0
    z: 0.0
  angular:
    x: 0.0
    y: 0.0
    z: 0.0"
```

Make the turtle move in a circle

```
$ rostopic pub -r 1 /turtle1/cmd_vel geometry_msgs/Twist -- '[2.0, 0.0, 0.0]' '[0.0,0.0, 1.8]'
```



Keyboard Control

```
$ rosrun turtlesim turtle_teleop_key
```

```
Reading from keyboard
---------------------------
Use arrow keys to move the turtle.
Up arrow
Turtle In Turtle’s x direction
Down arrow Turtle In Turtles’s -x direction
Right arrow Rotate CW
Left arrow Rotate CCW
```

```
$ rosnode list
```

```
/rosout
/teleop_turtle
/turtlesim
```

```
$ rosnode info /teleop_turtle
```

Node [/teleop_turtle]
Publications:
*** /turtle1/cmd_vel [geometry_msgs/Twist]**
* /rosout [rosgraph_msgs/Log]

The /teleop_turtle node is publishing on topic /turtle1/cmd_vel
Can you tell the message type for this topic ?

Subscriptions: None
Services:
* /teleop_turtle/get_loggers
* /teleop_turtle/set_logger_level

The /teleop_turtle node does not subscribe to any topic.

To clear the turtlesim screen use:

```
$ rosservice call /clear
```